登录 注册[Register]

网站 新帖 搜索 帮助 快捷导航

请输入搜索内容 搜索

网站 【 软件安全 】 『移动安全区』

返回列表 1 2 8 1 / 8 页

[Android 脱壳] 某加固的尝试（一） [复制链接]

straceme 2019-10-12 10:44

精华 楼主 电梯直达

本帖最后由 straceme 于 2019-10-12 11:17 编辑

1. 一次加固分析

由于要做测试，通过分析发现该应用加固了。该加固并不常见。尝试进行脱壳。

1.1. 样本信息

包名：com.xxx.jincheng
应用名称：某某银行
版本信息：3.0.8
MD5值：d0c5c54ca31ebcfa4834ebe7ecb23539

1.2. 初步分析

通过壳java层代码发现此加固的核心代码在libvdog.so中，其中libvdog.so文件的elf文件魔数被修改了， 壳代码会在写文件时将文件还原回去，如下：

```
         0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F    0123456789ABCDEF
0000h: 6E 61 67 61 01 01 01 00 00 00 00 00 00 00 00 00    naga.. .........
0010h: 03 00 28 00 01 00 00 00 00 00 00 00 34 00 00 00    ..(.........4...
0020h: 00 B0 16 00 00 00 00 05 34 00 20 00 08 00 28 00    .°......4. ...(.
0030h: 04 00 03 00 06 00 00 00 34 00 00 00 34 00 00 00    ........4...4...
0040h: 34 00 00 00 00 01 00 00 00 01 00 00 04 00 00 00    4...............
0050h: 04 00 00 00 03 00 00 00 34 01 00 00 34 01 00 00    ........4...4...
0060h: 34 01 00 00 13 00 00 00 13 00 00 00 04 00 00 00    4...............
0070h: 01 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00    ................
0080h: 00 00 00 00 00 60 06 00 00 60 06 00 05 00 00 00    .....`...`......
0090h: 00 10 00 00 01 00 00 00 00 60 06 00 00 60 06 00    .........`...`..
00A0h: 00 60 06 00 00 10 12 00 00 10 12 00 06 00 00 00    .`..............
00B0h: 00 10 00 00 02 00 00 00 F0 65 06 00 F0 65 06 00    ........ðe..ðe..
00C0h: F0 65 06 00 08 01 00 00 08 01 00 00 06 00 00 00    ðe..............
00D0h: 04 00 00 00 51 E5 74 64 00 00 00 00 00 00 00 00    ....Qåtd........
00E0h: 00 00 00 00 00 00 00 00 00 00 00 00 06 00 00 00    ................
00F0h: 00 00 00 00 01 00 00 70 80 1C 06 00 80 1C 06 00    .......p€...€...
0100h: 80 1C 06 00 C0 12 00 00 C0 12 00 00 04 00 00 00    €...À...À.......
0110h: 04 00 00 00 52 E5 74 64 90 53 06 00 90 63 06 00    ....Råtd.S...c..
0120h: 90 63 06 00 70 0C 00 00 70 0C 00 00 06 00 00 00    .c..p...p.......
0130h: 08 00 00 00 2F 73 79 73 74 65 6D 2F 62 69 6E 2F    ..../system/bin/
0140h: 6C 69 6E 6B 65 72 00 00 00 00 00 00 00 00 00 00    linker..........
0150h: 00 00 00 00 00 00 00 00 01 00 00 00 00 00 00 00    ................
0160h: 00 00 00 00 12 00 00 00 10 00 00 00 00 00 00 00    ................
0170h: 00 00 00 00 12 00 00 00 1D 00 00 00 29 C1 00 00    ............)Á..
0180h: 44 00 00 00 12 00 08 00 46 00 00 00 E5 7A 02 00    D.......F...åz..
0190h: 24 01 00 00 12 00 08 00 72 00 00 00 68 F1 05 00    $.......r...hñ..
01A0h: 08 00 00 00 12 00 08 00 89 00 00 00 6D C1 00 00    ........‰...mÁ..
01B0h: 40 00 00 00 12 00 08 00 A5 00 00 00 20 85 06 00    @.......¥... ….
01C0h: 04 00 00 00 11 00 13 00 CC 00 00 00 B1 C6 05 00    ........Ì...±Æ..
01D0h: 78 00 00 00 12 00 08 00 D2 00 00 00 E9 C1 00 00    x.......Ò...éÁ..
01E0h: 4C 01 00 00 22 00 08 00 EB 00 00 00 A9 D4 00 00    L..."...ë...©Ô..
01F0h: 94 00 00 00 22 00 08 00 09 01 00 00 00 00 00 00    ”..."...........
0200h: 00 00 00 00 12 00 00 00 1A 01 00 00 00 00 00 00    ................
0210h: 00 00 00 00 11 00 00 00 2C 01 00 00 31 A2 01 00    ........,...1¢..
0220h: 78 00 00 00 12 00 08 00 44 01 00 00 00 00 00 00    x.......D.......
0230h: 00 00 00 00 12 00 00 00 53 01 00 00 71 12 02 00    ........S...q...
0240h: E4 01 00 00 12 00 08 00 7C 01 00 00 D5 A7 01 00    ä.......|...Õ§..
0250h: 28 00 00 00 12 00 08 00 94 01 00 00 D5 E6 01 00    (.......”...Õæ..
0260h: C0 06 00 00 12 00 08 00 CD 01 00 00 91 CA 00 00    À.......Í...‘Ê..
0270h: 8C 00 00 00 22 00 08 00 ED 01 00 00 1D CB 00 00    Œ..."...í...Ë..
```

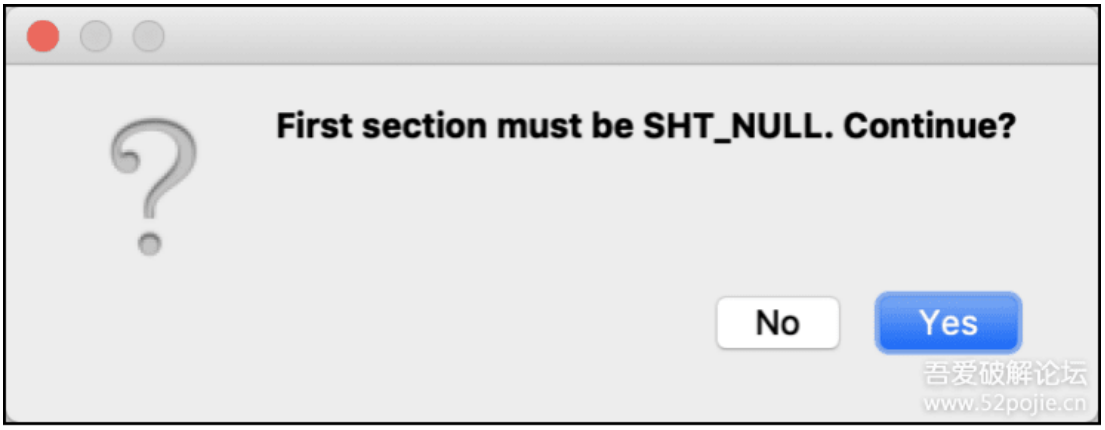
吾爱破解论坛 www.52pojie.cn

```
private static void a(String arg11, String arg12, Context arg13) {
    int v4;
    byte[] v7;
    int v3;
    BufferedOutputStream v1;
    BufferedInputStream v0;
    byte[] v2;
    try {
        InputStream v5 = arg13.getAssets().open(arg12);
        File v6 = new File(arg11);
        v2 = new byte[0x10000];
        v0 = new BufferedInputStream(v5);
        v1 = new BufferedOutputStream(new FileOutputStream(v6));
        v3 = 1;
        v7 = new byte[]{0x7F, 69, 76, 70};
        while(true) {
        label_27:
            v4 = v0.read(v2);
            if(v3 != 0) {
                break;
            }

            goto label_37;
        }
    }
    catch(Exception v8) {
        return;
    }

    v3 = 0;
    String v8_1 = ".so";
    try {
        if(arg11.endsWith(v8_1)) {
            System.arraycopy(v7, 0, v2, 0, v7.length);
        }

    label_37:
        if(v4 <= 0) {
            v1.flush();
            v1.close();
            v0.close();
        }
        else {
            v1.write(v2, 0, v4);
            goto label_27;
        }

        return;
    }
    catch(Exception v8) {
        return;
    }
}
```

这里对elf文件魔数进行了修改

吾爱破解论坛 www.52pojie.cn

直接用010editer把魔数改称 7F 45 4C 46 即可，此时文件libvdog.so可以使用IDA进行分析。在使用ida 打开文件过程中会发现，存在错误提示如下：



这说明libvdog.so的section信息已经被处理了，使用010Editer工具分析elf文件结构

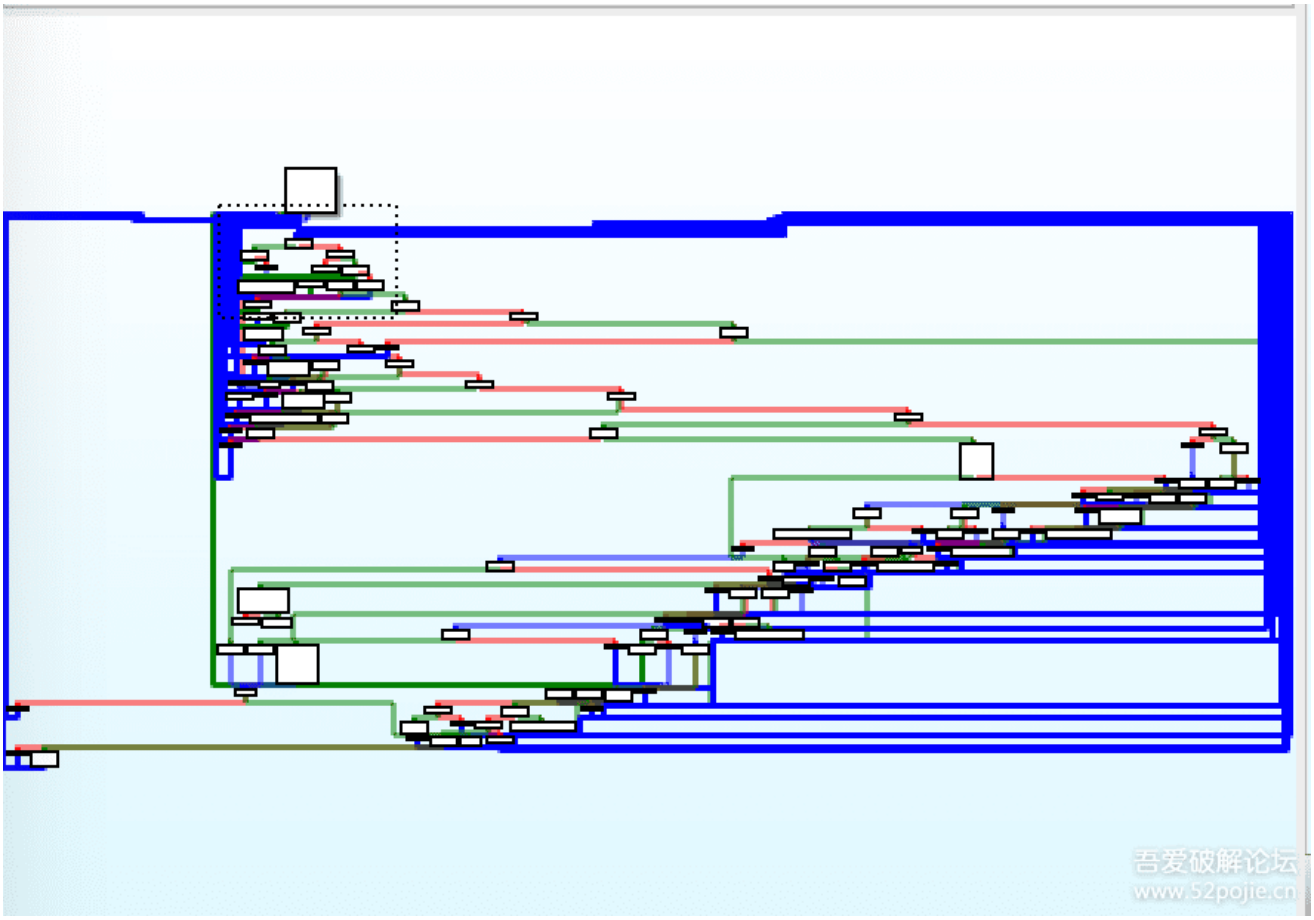
Name	Value	Start	Size	Color	
struct file		0h	666F8h	Fg: Bg:	
struct elf_header		0h	34h	Fg: Bg:	ELF_He
struct program_header_table		34h	100h	Fg: Bg:	
struct section_header_table		16B000h	A0h	Fg: Bg:	
struct section_table_entry32_...	SHN_UNDEF	16B000h	28h	Fg: Bg:	
struct section_table_entry32_...	SHN_UNDEF	16B028h	28h	Fg: Bg:	
struct section_table_entry32_...		16B050h	28h	Fg: Bg:	
struct section_table_entry32_...		16B078h	28h	Fg: Bg:	

吾爱破解论坛 www.52pojie.cn

但不影响分析。由于此动态库的导出函数可以判断，这so文件应该是一个自定义的linker文件。

Function name	Segment
crazy::Error::Format(char const*,...)	LOAD
crazy::FileDescriptor::SeekTo(long)	LOAD
crazy::ElfLoader::ReserveAddressSpace(crazy::Erro...	LOAD
crazy::ElfLoader::FindPhdr(crazy::Error *)	LOAD
crazy::ElfLoader::ReadProgramHeader(crazy::Error *)	LOAD
crazy::ElfLoader::LoadSegments(crazy::Error *)	LOAD
memcmp	**LOAD**
crazy::FileDescriptor::Read(void *,uint)	LOAD
crazy::FileDescriptor::Map(void *,uint,int,int,long)	LOAD
phdr_table_get_load_size(Elf32_Phdr const*,uint,uin...	LOAD
crazy::ElfLoader::CheckPhdr(uint,crazy::Error *)	LOAD
crazy::ElfRelocations::AdjustRelocation(uint,uint,uin...	LOAD
j__udivsi3	LOAD
crazy::ElfView::DynamicIterator::GetValue(void)	LOAD
crazy::ElfView::DynamicIterator::GetNext(void)	LOAD
crazy::ElfView::DynamicIterator::HasNext(void)	LOAD
crazy::ElfView::DynamicIterator::GetAddress(uint)	LOAD
crazy::Rand10(void)	LOAD
crazy::ElfView::DynamicIterator::DynamicIterator(cr...	LOAD
crazy::ElfRelocations::ApplyRelaRelocs(Elf32_Rela *...	LOAD
phdr_table_protect_segment(uint,uint,uint)	LOAD
phdr_table_unprotect_segments(Elf32_Phdr const*,...	LOAD
phdr_table_unprotect_segment(uint,uint,uint)	LOAD
crazy::ElfRelocations::ApplyRelRelocs(Elf32_Rel *,ui...	LOAD
phdr_table_protect_segments(Elf32_Phdr const*,int...	LOAD
crazy::ElfRelocations::ApplyAndroidRelocations(cra...	LOAD
crazy::ElfRelocations::ForEachAndroidRelocation(b...	LOAD
crazy::ElfRelocations::ApplyRelReloc(Elf32_Rel con...	LOAD
crazy::ElfRelocations::ApplyRelaReloc(Elf32_Rela c...	LOAD
crazy::AndroidPackedRelocationGroupFlags::hasF...	LOAD
crazy::Sleb128Decoder::pop_front(void)	LOAD
crazy::ElfRelocations::ResolveSymbol(uint,uint,craz...	LOAD

接下来就需要分析，真正的so代码数据存储在哪里，通过初步查看发现so函数已经混淆了。



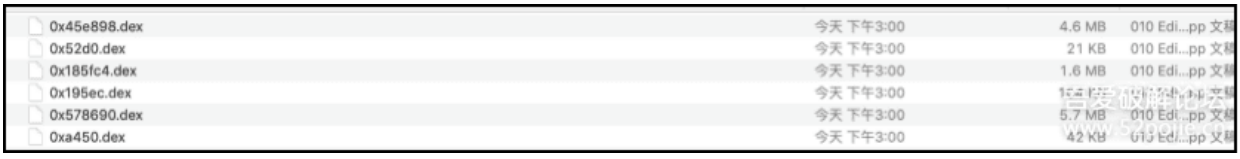
手动调试几乎不可能，几乎所以函数都进行了代码膨胀。
不过有一点还是可以庆幸的就是libvdog.so的字符串并未进行混淆，
详细分析该这so文件，代价太高了！！！，代先不进行分析，先进行整体dump。看看能拿到多少代码。

1. 3. dex文件整体dump

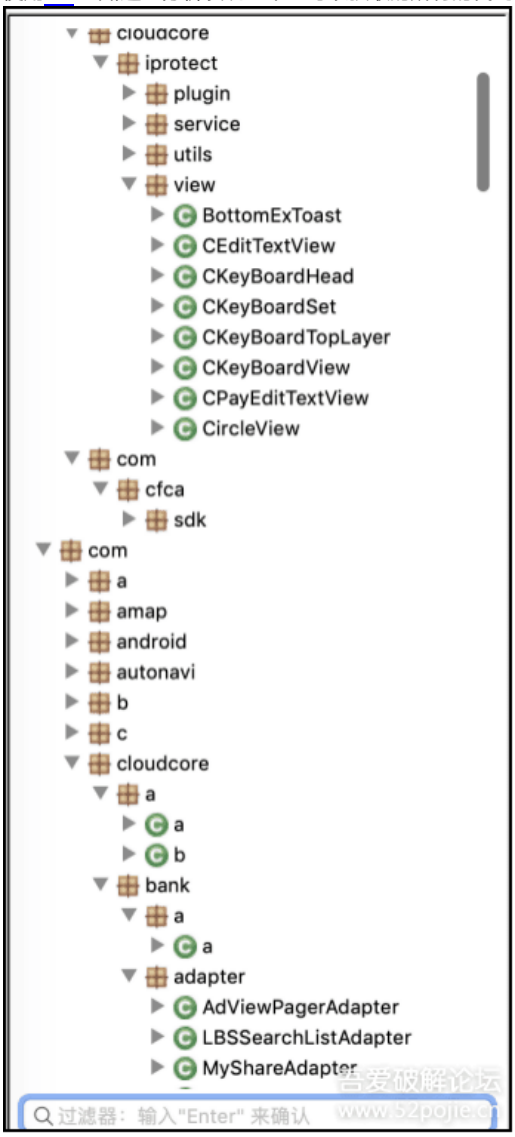
dex文件整体dump有很多办法，也有很多点，可以通过获取到所有dex的cookie，然后找到dex文件地址，也可以直接对关键函数进行拦截，我选择了DexFileLoader::OpenCommon这个点（android 10），其他版本的机器不确定有没有这个函数，直接hook。函数如下：

```
ifdef __LP64__

        std::string IsMagicValid_ = "_ZN3art15StandardDexFile12IsMagicValidEPKh";
        void *handle_ = SoInfo::GetHandle("libdexfile.so");
        void *IsMagicValid = dlsym(handle_, IsMagicValid_.c_str());
        WInlineHookFunction(IsMagicValid, (void *) &StandardDexFile_IsMagicValid_Q_x64,
                            (void **) &StandardDexFile_IsMagicValid_Q_x64_backup);

else
        std::string OpenCommon_ = "_ZN3art13DexFileLoader10OpenCommonEPKhjS2_jRKNSt3__112basic_strin
        void *handle_ = SoInfo::GetHandle( libpath: "libdexfile.so");
        void *OpenCommon = dlsym(handle_, OpenCommon_.c_str());
        WInlineHookFunction(OpenCommon, (void *) &DexFileLoader_OpenCommon_Q_x32,
                            (void **) &DexFileLoader_OpenCommon_Q_x32_backup);
endif

        break;
```

Dump 得到的dex文件如下：



使用JEB查看,通过分析发现基本上可以获取的所有的代码,




```
.method public onBackClick(View)V
        .registers 2
00000000  6E 10 50 85 00 00      invoke-virtual      GaodeLBSLocationAllActivity->finish()V, p0
00000006  0E 00                  return-void
.end method

.method public onCloudItemDetailSearched(CloudItemDetail, I)V
        .registers 3
00000000  0E 00                  return-void
.end method

.method public onCloudSearched(CloudResult, I)V
        .registers 17
00000000  6E 10 6C 76 0F 00      invoke-virtual      CloudResult->getClouds()ArrayList, p1
00000006  0C 03                  move-result-object  v3
00000008  22 04 89 0C            new-instance        v4, MarkerOptions
0000000C  70 10 15 6A 04 00      invoke-direct       MarkerOptions-><init>()V, v4
00000012  22 05 26 13            new-instance        v5, JSONObject
00000016  70 10 27 95 05 00      invoke-direct       JSONObject-><init>()V, v5
0000001C  6E 10 7F 91 03 00      invoke-virtual      ArrayList->iterator()Iterator, v3
00000022  0C 06                  move-result-object  v6
:24
00000024  72 10 ED 91 06 00      invoke-interface    Iterator->hasNext()Z, v6
0000002A  0A 00                  move-result         v0
0000002C  38 00 91 00            if-eqz              v0, :14E
:30
00000030  72 10 EE 91 06 00      invoke-interface    Iterator->next()Object, v6
00000036  0C 00                  move-result-object  v0
00000038  1F 00 ED 0D            check-cast          v0, CloudItem
```

描述 | 十六进制格式 | Disassembly | Graph | 字符串

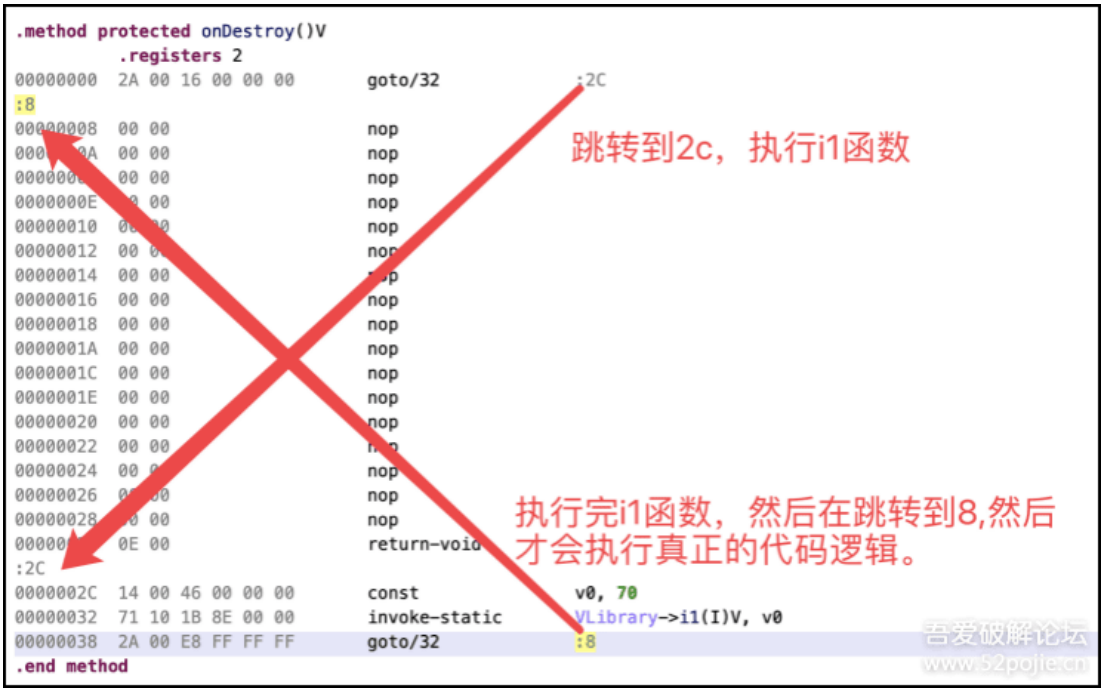
到这里基本上就可以正常分析了,
通过查看发现。还有一部分代码不被抽取了。大概特征如下:

```
.method protected onDestroy()V
        .registers 2
00000000  2A 00 16 00 00 00      goto/32            :2C
:8
00000008  00 00                  nop
0000000A  00 00                  nop
0000000C  00 00                  nop
0000000E  00 00                  nop
00000010  00 00                  nop
00000012  00 00                  nop
00000014  00 00                  nop
00000016  00 00                  nop
00000018  00 00                  nop
0000001A  00 00                  nop
0000001C  00 00                  nop
0000001E  00 00                  nop
00000020  00 00                  nop
00000022  00 00                  nop
00000024  00 00                  nop
00000026  00 00                  nop
00000028  00 00                  nop
0000002A  0E 00                  return-void
:2C
0000002C  14 00 46 00 00 00      const              v0, 70
00000032  71 10 1B 8E 00 00      invoke-static      VLibrary->i1(I)V, v0
00000038  2A 00 E8 FF FF FF      goto/32            :8
.end method
```

吾爱破解论坛 www.52pojie.cn

1.4. 抽取函数破解尝试

怎么还原这些被抽取的函数。尝试在工具dexhunter,然并卵。并没有什么用处。代码逻辑还是没还原。尝试分析其smali代码逻辑。



猜测，通过i1函数进行还原。上面nop代码，然后在执行真正代码逻辑。这样的好处，当运行到某个函数时候代码才会还原。关键点在i1函数中，可以通过两个方面进行解决

1. 直接分析i1函数，然后进行还原
2. 使用开源方案开源的方案。

通过虚拟机主动加载类函数。是个不错的思路。https://bbs.pediy.com/thread-252630.htm可以直接按照这个思路，
对虚拟机进行修改。**具体的细节后面整理完在开源**，坑有点多还原前后代码对比
还原前：

```
.method protected onDestroy()V
        .registers 2
00000000  2A 00 16 00 00 00        goto/32            :2C
:8
00000008  00 00                    nop
0000000A  00 00                    nop
0000000C  00 00                    nop
0000000E  00 00                    nop
00000010  00 00                    nop
00000012  00 00                    nop
00000014  00 00                    nop
00000016  00 00                    nop
00000018  00 00                    nop
0000001A  00 00                    nop
0000001C  00 00                    nop
0000001E  00 00                    nop
00000020  00 00                    nop
00000022  00 00                    nop
00000024  00 00                    nop
00000026  00 00                    nop
00000028  00 00                    nop
0000002A  0E 00                    return-void
:2C
0000002C  14 00 46 00 00 00        const              v0, 70
00000032  71 10 1B 8E 00 00        invoke-static      VLibrary->i1(I)V, v0
00000038  2A 00 E8 FF FF FF        goto/32            :8
.end method
```

还原前

还原后：

```
.method protected onDestroy()V
        .registers 2
00000000  2A 00 16 00 00 00        goto/32            :2C
:8
00000008  6F 10 82 83 01 00        invoke-super       BaseActivity->onDestroy()V, p0
0000000E  54 10 F8 43              iget-object        v0, p0, GaodeLBSLocationAllActivity->b:MapView
00000012  6E 10 06 68 00 00        invoke-virtual     MapView->onDestroy()V, v0
00000018  54 10 FC 43              iget-object        v0, p0, GaodeLBSLocationAllActivity->f:AMapLocationClient
0000001C  38 00 07 00              if-eqz             v0, :2A
:20
00000020  54 10 FC 43              iget-object        v0, p0, GaodeLBSLocationAllActivity->f:AMapLocationClient
00000024  6E 10 AD 62 00 00        invoke-virtual     AMapLocationClient->onDestroy()V, v0
:2A
0000002A  0E 00                    return-void
:2C
0000002C  14 00 46 00 00 00        const              v0, 70
00000032  71 10 1B 8E 00 00        invoke-static      VLibrary->i1(I)V, v0
00000038  2A 00 E8 FF FF FF        goto/32            :8
.end method
```

还原后

直接分析i1的方式还没时间完成分析。工作量太大。

1.5. 自定义dex解释器代码还原

搜索发现，所有的onCreate方法代码逻辑都被替换掉了，开始以为是抽取的后来发现不是。
代码如下：

```
.method protected onCreate(Bundle)V
        .registers 9
00000000  13 01 03 00              const/16           v1, 3
00000004  23 10 6F 14              new-array          v0, v1, [Object
00000008  13 01 00 00              const/16           v1, 0
0000000C  4D 07 00 01              aput-object        p0, v0, v1
00000010  13 01 01 00              const/16           v1, 1
00000014  4D 08 00 01              aput-object        p1, v0, v1
00000018  14 02 14 00 00 00        const              v2, 20
0000001E  71 10 59 8F 02 00        invoke-static      Integer->valueOf(I)Integer, v2
00000024  0C 02                    move-result-object v2
00000026  13 01 02 00              const/16           v1, 2
0000002A  4D 02 00 01              aput-object        v2, v0, v1
0000002E  71 10 1C 8E 00 00        invoke-static      VLibrary->v1([Object)Object, v0
00000034  0E 00                    return-void
.end method
```

通过hook 简单验证了下；
发现当运行时会调用JNI(Java Native Interface)的接口执行代码逻辑。
由于分析详细过程可能会涉及的加固的核心利益，暂不进行分享。
大概思路，使用libdexfile这个库，对dex文件进行拆分，然后新建一个codeItem类

```
for (const auto &code_items: mHeader->GetCollections().CodeItems()) {
    const char *method_name = code_items->GetMethodItem()->GetMethodId()->Name()->Data();
    if (strstr(method_name, "onCreate") != nullptr) {
        LOGI("[+]fix %s %s", code_items->GetMethodItem()->GetClassName().c_str(), method_name);
        uint32_t index = getOncreateIndex(code_items.get());
        if (index == -1) {
            LOGE("[-] skip fix %s vmp code", method_name);
            continue;
        }
        LOGI("get vmp index:%x", index);
        auto dexCode = vdogList->get_vmp_code_by_index(index);

        dexCode->debugInfoOff = 0;
        dex_ir::CodeItem *codeItem = mHeader->GetCollections().CreateCodeItem( dex_file: *mDexFile.get(),
                                                                                reinterpret_cast<uint8_t *>(dexCode),
                                                                                index,
                                                                                code_items->GetMethodItem()->GetMethodId()->GetIndex());
        code_items->GetMethodItem()->SetCodeItem( &: codeItem);
        repairVMPCode(codeItem);
    }
}
```

对代码进行还原。
中间涉及了太多。后面有机会进行分享。还原后的代码如下：

```
protected void onCreate(Bundle arg3) {
    super.onCreate(arg3);
    this.requestWindowFeature(1);
    this.setContentView(0x7F090027);
    this.a = ((Context)this);
    this.b = this.findViewById(0x7F07010E);
    this.b.onCreate(arg3);
    this.l = this.getIntent().getStringExtra("funName");
    this.m = this.getIntent().getStringExtra("UserInfo");
    this.a();
}

protected void onDestroy() {
    VLibrary.i1(70);
    super.onDestroy();
    this.b.onDestroy();
    if(this.f != null) {
        this.f.onDestroy();
    }
}
```

1.6. 修复后的dex下载

下载: https://www.lanzous.com/i6pnm1c 密码:4jxk
由于时间问题，很多的细节，都没写，后面将会将具体细节分享出来。

2. 参考以及使用工具

2.1. 代码

- Android Open Source Project(libdexfile) https://android.googlesource.com/platform/art/+/master/libdexfile/
- PLT hook: https://github.com/iqiyi/xHook
- inlineHook https://github.com/jmpews/HookZz

2.2. 工具

- IDA Pro
- Clion
- 010editer

2.3. 脚本

- 010Editer elf文件脚本：https://raw.githubusercontent.com/strazzere/010Editor-stuff/master/Templates/ELFTemplate.bt

免费评分

吾爱币	热心值	理由	收起
	+ 1	我很赞同！	

waterwelly			
zuoking991	+ 1	+ 1	谢谢@Thanks!
bianfanggege	+ 1	+ 1	谢谢
云上天堂		+ 1	支持，感谢
鹤舞九月天	+ 1	+ 1	谢谢@Thanks!
0xxx	+ 1	+ 1	我很赞同!
糖粒er	+ 1	+ 1	谢谢@Thanks!
不爱everyone	+ 1	+ 1	感谢发布原创作品，吾爱破解论坛因你更精彩!
hjw45611	+ 1	+ 1	热心回复!
Spareks	+ 1	+ 1	用心讨论，共获提升!
you920928	+ 1	+ 1	谢谢@Thanks!
yaoyao7	+ 1	+ 1	我很赞同!
安尼大大	+ 1	+ 1	我很赞同!
N0LL	+ 1	+ 1	谢谢@Thanks!
llm4041	+ 1	+ 1	看蒙了,真牛批
飞龙Test	+ 1	+ 1	谢谢@Thanks!
JuDei	+ 1	+ 1	热心回复!
禾水木	+ 1	+ 1	热心回复!
lookerJ	+ 1	+ 1	用心讨论，共获提升!
fcguo800	+ 2	+ 1	没得说，关注支持大佬分享。
sunnylds7	+ 1	+ 1	热心回复!
AnonymousQsh	+ 1	+ 1	用心讨论，共获提升!
丶咖啡猫丶	+ 1	+ 1	谢谢@Thanks!
thinkpad_420	+ 1	+ 1	我很赞同!
shun	+ 1	+ 1	plus
莫流云	+ 1	+ 1	感谢发布原创作品，吾爱破解论坛因你更精彩!
dreamlivemeng	+ 1	+ 1	用心讨论，共获提升!
一人之下123456		+ 1	用心讨论，共获提升!
XhyEax	+ 3	+ 1	热心回复!
mrsdz	+ 1	+ 1	热心回复!
jiajs	+ 1	+ 1	我很赞同!
abc2342537	+ 1	+ 1	我很赞同!
Gentlewang	+ 3	+ 1	支持一下技术大佬!
cxp521	+ 1	+ 1	流弊，流弊

查看全部评分

本帖被以下淘专辑推荐:

· **安卓及破解案例** | 主题: 355, 订阅: 224

收藏 54 淘帖 1

发帖前要善用【**论坛搜索**】功能，那里可能会有你要找的答案或者已经有人发布过相同内容了，请勿重复发帖。

回复 举报

Hmily 2019-10-12 12:59 推荐

看那个运行到加密地方解码后再执行，完事再加密很像windows时期很多壳如TMD的sdk里那种SMC的加密方式，看起来都照搬到安卓里了。

【吾爱破解论坛总版规】 - [让你充分了解吾爱破解论坛行为规则]

回复 支持 6 举报

straceme 2019-10-15 09:44 推荐

吾爱破解论坛没有任何官方QQ群，禁止留联系方式，禁止任何商业交易。

> 雪一梦 发表于 2019-10-12 15:00
> 楼主你好：
> “使用libdexfile这个库，对dex文件进行拆分，然后新建一个codeItem类”
> 这一块我看repairVMP ...

libdexfile 将dex文件拆分开，然后通过repairVMPCode 进行修复，后面有空将文章发出来，这里直接将文件解密后拿到的。

如何升级？如何获得积分？积分对应解释说明！

回复 支持 举报

cxp521 2019-10-12 11:00 4#

《站点帮助文档》有什么问题来这里看看吧，这里有你想知道的内容！

流弊，流弊，杠杠的

呼吁大家发布原创作品添加吾爱破解论坛标示！

回复 支持 举报

Vincent-HJ 2019-10-12 11:30 5#

虽然看不懂，但是觉得很流弊~

如何快速判断一个文件是否为病毒！

回复 支持 举报

thinkpad_420 2019-10-12 11:38 6#

好厉害，算是开眼界了

回复 支持 举报

zg2600 2019-10-12 11:40 7#

我成功了呢

回复 支持 举报

Rolanju 2019-10-12 11:46 8#

先收藏再说

回复 支持 举报

风绕柳絮轻敲雪 2019-10-12 11:53 9#

爱加密？

回复 支持 举报

shelly1314 2019-10-12 12:28 10#

感谢楼主分享

回复 支持 举报

一人之下123456 2019-10-12 12:51 11#

感觉很厉害的样子，向大佬学习。

回复 支持 举报

duduhao 2019-10-12 12:52 12#

学习了，谢谢楼主的好分享

回复 支持 举报

下一页 »

返回列表 1 2 8 1 / 8 页

高级模式

您需要登录后才可以回帖 登录 | 注册[Register] 用QQ帐号登录

发表回复 警告：本版块禁止灌水或回复与主题无关内容，违者重罚！ 回帖并转播 回帖后跳转到最后一页

免责声明：
吾爱破解所发布的一切破解补丁、注册机和注册信息及软件的解密分析文章仅限用于学习和研究目的；不得将上述内容用于商业或者非法用途，否则，一切后果请用户自负。本站信息来自网络，版权争议与本站无关。您必须在下载后的24个小时之内，从您的电脑中彻底删除上述内容。如果您喜欢该程序，请支持正版软件，购买注册，得到更好的正版服务。如有侵权请邮件与我们联系处理。

Mail To:Service@52PoJie.Cn